Artificial Intelligence
Lecture 6: Knowledge-based agents and propositional logic

Prof. Dr. Pierre-Alexandre Murena

SoSe 2025

Outline

1 Knowledge-based agents

2 What is logic

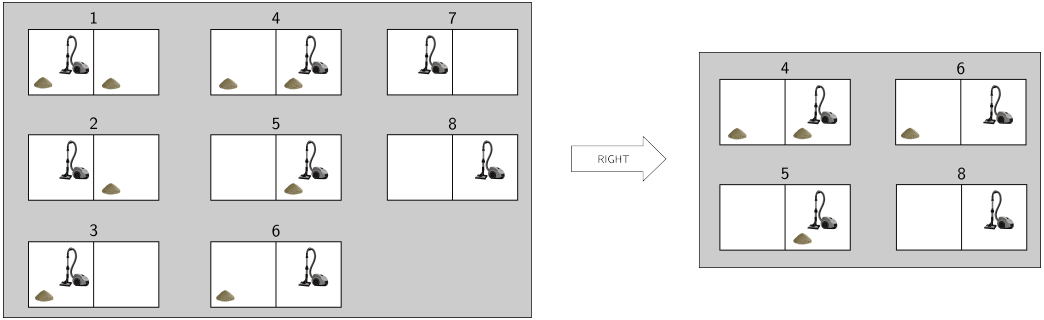
3 Propositional logic

Plan

1. Knowledge-based agents
   - Knowledge and belief filtering
   - Knowledge-based agents
2. What is logic
3. Propositional logic

Reminder: Belief states

Blind setting: The agent does not perceive anything but can act.



Representing physical states

A (physical) state contains two elements:

- The position of the agent
- The location of dust

We can represent it as a vector of 4 Boolean values:

$$is_left, is_right, dust_left, dust_right$$

Constraints:

- The agent is at one location at a time
- No constraint on the dust

Representing belief states

In the initial belief state, the agent ignores everything about the world: it has **no knowledge**.

After moving right, the agent can be in 4 different physical states, having all in common that

$$is_right = \text{True}$$

Representing belief states

In the initial belief state, the agent ignores everything about the world: it has **no knowledge**.

After moving right, the agent can be in 4 different physical states, having all in common that

$$is_right = \text{True}$$

Similarly, the agent can be in 4 different physical states after moving left, having in common:

$$is_left = \text{True}$$

Representing belief states

In the initial belief state, the agent ignores everything about the world: it has **no knowledge**.

After moving right, the agent can be in 4 different physical states, having all in common that

$$is_right = \text{True}$$

Similarly, the agent can be in 4 different physical states after moving left, having in common:

$$is_left = \text{True}$$

If the agent is in the right cell and cleans, the physical states in the new belief states have in common that:

$$is_right = \text{True} \quad \textbf{and} \quad dust_right = \text{False}$$

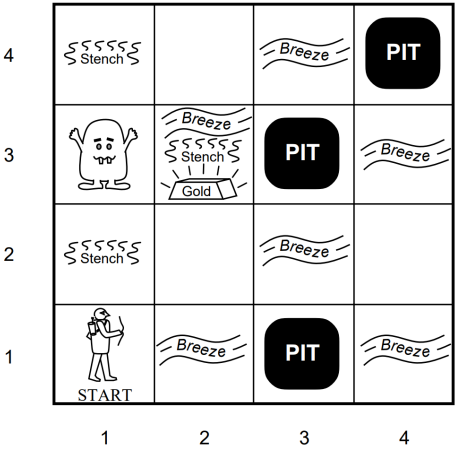
Knowledge base

These representations of what the agent knows about the world are called **knowledge bases**.

A knowledge base is a set of **sentences**, expressed in a **knowlege representation language**, which can describe:

- **Background knowledge:** knowledge about the environment, prior to any interaction. *E.g. the agent is at one location at a time.*
- **Acquired knowledge:** knowledge acquired through direct sensing in the environment. *E.g. in case of partial observability, seeing dust in the agent's cell.*
- **Inferred knowledge:** knowledge acquired indirectly, through reasoning about percepts. *E.g. the agent moved right, so must be in the right cell (independently of its initial position).*

A more complicated example: The Wumpus world



In cells adjacent to the Wumpus, the agent perceives a stench.

In cells adjacent to a pit, the agent perceives a breeze.

The agent starts in (1,1) and is aware of its location.

The goal of the agent is to reach the gold and to avoid the Wumpus and the pits.

Wumpus world: Step 1

The agent is initially in cell (1, 1) and perceives nothing. What does this imply?

- The Wumpus is not in (1,1)
- No pit in (1,1)

Wumpus world: Step 1

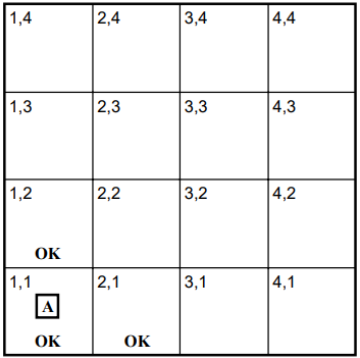
The agent is initially in cell (1, 1) and perceives nothing. What does this imply?

- The Wumpus is not in (1,1)
- No pit in (1,1)
- The Wumpus is neither in (1,2) nor in (2,1).
- No pit in (1,2) and in (2,1).

Wumpus world: Step 1

The agent is initially in cell (1, 1) and perceives nothing. What does this imply?

- The Wumpus is not in (1,1)
- No pit in (1,1)
- The Wumpus is neither in (1,2) nor in (2,1).
- No pit in (1,2) and in (2,1).



Wumpus world: Step 2

The agent moves to cell (2, 1) and perceives a breeze. What does this imply?

- The Wumpus is neither in (2,2) nor in (3,1).
- Pit either in (2,2) and in (3,1).

Wumpus world: Step 2

The agent moves to cell (2, 1) and perceives a breeze. What does this imply?

- The Wumpus is neither in (2,2) nor in (3,1).
- Pit either in (2,2) and in (3,1).

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

Wumpus world: Step 3

Since it's not safe to move to (2,2) and to (3,1), the agent moves to cell (1,2) and perceives a stench. What does this imply?

- The Wumpus is either in (1,3) or in (2,2).
- Pit in neither in (1,3) nor in (2,2).

Wumpus world: Step 3

Since it's not safe to move to (2,2) and to (3,1), the agent moves to cell (1,2) and perceives a stench. What does this imply?

- The Wumpus is either in (1,3) or in (2,2).
- Pit in neither in (1,3) nor in (2,2).
- If the Wumpus were in (2,2), a stench would have been perceived in step 2.
  Consequently, the Wumpus is in (1,3).

Wumpus world: Step 3

Since it's not safe to move to (2,2) and to (3,1), the agent moves to cell (1,2) and perceives a stench. What does this imply?

- The Wumpus is either in (1,3) or in (2,2).
- Pit in neither in (1,3) nor in (2,2).
- If the Wumpus were in (2,2), a stench would have been perceived in step 2.
  Consequently, the Wumpus is in (1,3).

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

Plan

1 Knowledge-based agents
- Knowledge and belief filtering
- Knowledge-based agents

2 What is logic

3 Propositional logic

Knowledge-based agents

Knowledge-based agents rely on some knowledge about the world to choose actions and make decisions.

Knowledge corresponds to assertions about the world, which are **represented** in the form of sentences in a formal language, called **representation language**.

Knowledge-based agents

Knowledge-based agents rely on some knowledge about the world to choose actions and make decisions.

Knowledge corresponds to assertions about the world, which are **represented** in the form of sentences in a formal language, called **representation language**.

A knowledge-based agent is composed of:

- A **knowledge based**: containing all the assertions about the world known by the agent (*domain specific*);
- An **inference mechanism**: describing how the agent reasons to deduce hidden properties of the world (*domain independent*).

Characteristics of knowledge-based agents

A knowledge-based agent is capable of:

- Representing the problem, in particular states and actions;
- Incorporating new percepts;
- Updating *internal* representations of the world;
- Deducing hidden properties of the world;
- Deducing optimal actions accordingly.

Declarative method for building knowledge-based agents

A knowledge-based agent is interfaced with two main methods:

TELL: used to tell the agent what it needs to know.

ASK: used to ask the agent what to do (following its knowledge).

```
function KB-AGENT(percept) returns an action
   static: KB, a knowledge base
           t, a counter, initially 0, indicating time

   TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
   action ← ASK(KB, MAKE-ACTION-QUERY(t))
   TELL(KB, MAKE-ACTION-SENTENCE(action, t))
   t ← t + 1
   return action
```

Outline

1 Knowledge-based agents

2 What is logic

3 Propositional logic

Knowledge representation and reasoning

When designing a knowledge-based agent, an important challenge is to choose the appropriate **representation language** and the optimal **inference method**.

Logics are formal languages used to **represent knowledge** and to **extract solutions**.

How to define a language formally?

Syntax and semantics

Formal languages are formally defined by two sets of rules:

- **Syntax:** Rules that dictate which sentences are well formed.
  Example: The English sentence "the is and" is syntactically incorrect.

- **Semantics:** Describes the *meaning* of a sentence.
  Example: The English sentence "The dancing toaster serenaded the moon with its buttery lamps" is syntactically correct, but semantically incorrect.

Plan

1 Knowledge-based agents

2 What is logic
- Models and entailment
- Inference

3 Propositional logic

Semantics in logic: Truth

In logic, the semantic of the language defines the **truth** of sentences.

Traditionally, the truth can have a binary value: either the sentence is true or it is false. (*This is not the case in fuzzy logic, though*).

Semantics in logic: Truth

In logic, the semantic of the language defines the **truth** of sentences.

Traditionally, the truth can have a binary value: either the sentence is true or it is false. (*This is not the case in fuzzy logic, though*).

Truth is evaluated relatively to **all** possible worlds.

For instance, in a belief state, a property is true only if it is true for **all** physical states in the belief state.

Models

A **model** is a structured world with respect to which truth can be evaluated.

m is a model of a sentence α if α is true in m.

Models

A **model** is a structured world with respect to which truth can be evaluated.

m is a model of a sentence α if α is true in m.

Example 1: The following physical state is a model of the sentence "The right cell is clean".

Models

A **model** is a structured world with respect to which truth can be evaluated.

m is a model of a sentence α if α is true in m.

Example 1: The following physical state is a model of the sentence "The right cell is clean".

Example 2: $x = 1$ and $y = 3$ is a model of the sentence "$x + y = 4$".

Models

A **model** is a structured world with respect to which truth can be evaluated.

m is a model of a sentence α if α is true in m.

Example 1: The following physical state is a model of the sentence "The right cell is clean".

Example 2: $x = 1$ and $y = 3$ is a model of the sentence "$x + y = 4$".

Remark: We denote by $M(\alpha)$ the set of all models of α.

Entailment

A knowledge base KB is said to **entail** sentence α if and only if all models of KB are also models of α:

$$M(KB) \subseteq M(\alpha)$$

We then write:

$$KB \models \alpha$$

Example: Entailment in the Wumpus world

At step 2 of the Wumpus game, the knowledge base KB is the following:

Rules of the Wumpus world
(1,1) and (2,1) are empty
Breeze in (2,1)

This can be summarized by the following description of the world:

?	?	
OK	OK *breeze*	?

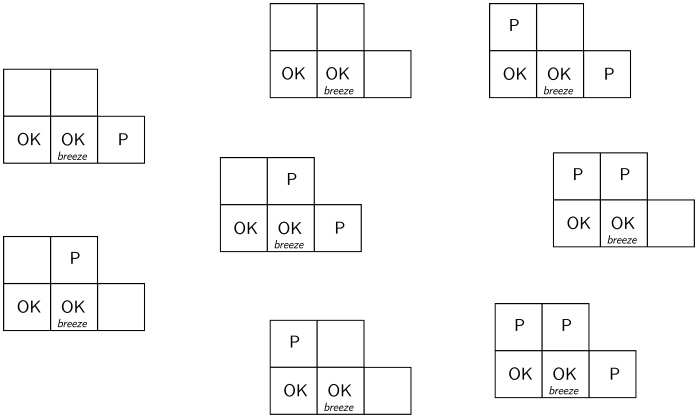
Example: Entailment in the Wumpus world

Trying to locate the pits, we obtain 8 possible worlds:
This can be summarized by the following description of the world:

?	?	
OK	OK *breeze*	?

Example

We have 8 possible configurations for the world:



Example

Let $\alpha_1 =$ "(1,2) is safe". Let's show that $KB \models \alpha_1$.

Let $\alpha_2 =$ "(2,2) is safe". Let's show that $KB \models \alpha_1$. Do we have "(2,2) is safe"?

Plan

1 Knowledge-based agents

2 What is logic
- Models and entailment
- Inference

3 Propositional logic

Why entailment is not enough

For an agent, it is not enough that the knowledge base entails some sentence α: Entailment describes some absolute truth, but not necessarily a truth that the agent can perceive.

Example: In the language of mathematics, for all $r \geq 3$, we have $a^r + b^r = c^r$ with a, b, c natural numbers imply $a = b = c$. This result is known to be true, but for a long time was not proved, i.e. was not accessible to mathematicians.

As a consequence, we need to introduce the way an agent draws new conclusions from a knowledge base.

Inference

Inference is a mechanism used to derive sentences from other sentences.

We use the notation

$$KB \vdash_i \alpha$$

to say that the sentence α is **derived** from KB by inference procedure i.

Sound inference procedures

A trivial inference procedure would consist in deriving one unique α_0 from any possible knowledge base.

If α_0 is not true, then the inference procedure derives a false result. This is obviously a behaviour we want to avoid.

Sound inference procedures

A trivial inference procedure would consist in deriving one unique α_0 from any possible knowledge base.

If α_0 is not true, then the inference procedure derives a false result. This is obviously a behaviour we want to avoid.

An inference procedure i is called **sound** if, whenever $KB \vdash_i \alpha$, we have $KB \models \alpha$.

Complete inference procedures

A trivial inference procedure would consist in deriving one unique α_0 from any possible knowledge base.

In general, the knowledge base entails more than only α_0. In this case, the inference procedure "misses" some true sentences.

Complete inference procedures

A trivial inference procedure would consist in deriving one unique α_0 from any possible knowledge base.

In general, the knowledge base entails more than only α_0. In this case, the inference procedure "misses" some true sentences.

An inference procedure i is called **complete** if, whenever $KB \models \alpha$, we have $KB \vdash_i \alpha$.

Outline

1 Knowledge-based agents

2 What is logic

3 Propositional logic

Motivation

We now introduce **propositional logic**, which is one specific simple logic. We will show that this logic is expressive enough to describe the Wumpus world for instance.

Plan

1 Knowledge-based agents

2 What is logic

3 Propositional logic
- Syntax
- Semantics
- Logical equivalence
- Propositional inference: Enumeration method

Propositions

A **proposition** is a declarative statement that is either true or false. Propositions are the sentences of the propositional logic $\mathcal{PL}$.

A proposition can be:

- **An atomic proposition**: this corresponds to propositional symbols: For instance $P, Q, isGreen$
- **A compound proposition**: constructed from other propositions (atomic or compound) using **logical connectives**.

Remark: The names of the propositional symbols are often chosen in the context of specific problems to reflect actual aspects of the world expressed in natural language. It is important to keep in mind that they are just symbols.

Logical connectives

Compound propositions are built from other propositions (atomic or compound) and 5 logical connectives:

- **Negation** ($\neg$): Unary connector: $\neg(p)$
- **Conjunction** ($\wedge$): Binary connector: $p \wedge q$
- **Disjunction** ($\vee$): Binary connector: $p \vee q$
- **Implication** ($\rightarrow$): Binary connector: $p \rightarrow q$
- **If and only if** ($\leftrightarrow$): Binary connector: $p \leftrightarrow q$

Remark: At this stage the connectors don't have any semantic meaning.

Backus-Naur Form of the grammar

$$
\begin{aligned}
< Proposition > &\rightarrow < AtomicSentences > \mid < CompoundSentences > \\
< AtomicSentences > &\rightarrow < PropositionalSymbols > \\
< CompoundSentences > &\rightarrow \neg(Proposition) \\
&\quad \mid (< proposition >< binaryconnector >< proposition >) \\
< binaryconnector > &\rightarrow \wedge \mid \vee \mid \rightarrow \mid \leftrightarrow
\end{aligned}
$$

Remark: Formulas and interpretations

Some properties seem familiar to us. For instance, we are used to having $a \wedge b$ equal to $b \wedge a$.

For now, this is **incorrect**: the two formulas are only strings of symbols, and consequently are equal only if the strings are equal.

Remarks:

- You can see an analogy with natural language. If you consider the character strings only, "Hamburg" and "Hansestadt Hamburg" are different. They are equal only when you consider the semantics, i.e. the meaning of the strings.
- The fact that the property "$a \wedge b$ is equal to $b \wedge a$" seems intuitive is a consequence of this choice of notations. Had we used other symbols, it wouldn't be obvious. For instance $\xi\kappa\varrho$ and $\varrho\kappa\xi$.
- For this reason, the parentheses matter! In the syntax, the formula $p_1 \wedge (p_2 \wedge p_3)$ is valid, but not $p_1 \wedge p_2 \wedge p_3$.

Plan

1 Knowledge-based agents

2 What is logic

3 Propositional logic
- Syntax
- Semantics
- Logical equivalence
- Propositional inference: Enumeration method

Semantics in model theory

Propositional symbols are atomic sentences of $\mathcal{PL}$: therefore a natural way is to attribute them a truth value, called an **interpretation**.

For all propositional variable x, we use the notations $\mathcal{I}(x)$, or $x^{\mathcal{I}}$ for an interpretation of x.

Semantics in model theory

Propositional symbols are atomic sentences of $\mathcal{PL}$: therefore a natural way is to attribute them a truth value, called an **interpretation**.

For all propositional variable x, we use the notations $\mathcal{I}(x)$, or $x^{\mathcal{I}}$ for an interpretation of x.

An interpretation of a compound proposition is defined by giving an interpretation to connectors.

Truth value of connectors

$$(\neg\phi)^{\mathcal{I}} = \begin{cases} \mathit{false} & \text{if } \phi^{\mathcal{I}} = \mathit{true} \\ \mathit{true} & \text{otherwise} \end{cases}$$

$$(\phi_1 \wedge \phi_2)^{\mathcal{I}} = \begin{cases} \mathit{true} & \text{if } \phi_1^{\mathcal{I}} = \phi_2^{\mathcal{I}} = \mathit{true} \\ \mathit{false} & \text{otherwise} \end{cases}$$

$$(\phi_1 \vee \phi_2)^{\mathcal{I}} = \begin{cases} \mathit{false} & \text{if } \phi_1^{\mathcal{I}} = \phi_2^{\mathcal{I}} = \mathit{false} \\ \mathit{true} & \text{otherwise} \end{cases}$$

$$(\phi_1 \rightarrow \phi_2)^{\mathcal{I}} = \begin{cases} \mathit{true} & \text{if } \phi_1^{\mathcal{I}} = \mathit{false} \text{ or } \phi_2^{\mathcal{I}} = \mathit{true} \\ \mathit{false} & \text{otherwise} \end{cases}$$

$$(\phi_1 \leftrightarrow \phi_2)^{\mathcal{I}} = \begin{cases} \mathit{true} & \text{if } \phi_1^{\mathcal{I}} = \phi_2^{\mathcal{I}} \\ \mathit{false} & \text{otherwise} \end{cases}$$

Models and interpretations

A **model** of ϕ is an interpretation of ϕ which satisfies ϕ (i.e. $\mathcal{I}$ such that $\phi^{\mathcal{I}} = \mathit{true}$).

Example: The interpretation $\mathcal{I} = a\bar{b}$ is a model of $a \vee b$.
The models of $a \vee b$ are $\mathcal{M}(a \vee b) = \{ab, a\bar{b}, \bar{a}b\}$.

Models and interpretations

A **model** of ϕ is an interpretation of ϕ which satisfies ϕ (i.e. $\mathcal{I}$ such that $\phi^{\mathcal{I}} = \mathit{true}$).

Example: The interpretation $\mathcal{I} = a\bar{b}$ is a model of $a \vee b$.
The models of $a \vee b$ are $\mathcal{M}(a \vee b) = \{ab, a\bar{b}, \bar{a}b\}$.

Now that we have defined the models, we have also defined the entailment $\models$. This corresponds to defining the semantics of $\mathcal{PL}$. We use the notation $(\mathcal{PL}, \models)$ to designate the propositional logic endowed with this proposed semantic.

Computing the interpretation of a compound proposition

We can compute the interpretation of any proposition recursively:

Computing the interpretation of a compound proposition

We can compute the interpretation of any proposition recursively:

Complexity 0: Propositional symbols are propositions of complexity 0. Their interpretation has been defined already.

Computing the interpretation of a compound proposition

We can compute the interpretation of any proposition recursively:

Complexity 0: Propositional symbols are propositions of complexity 0. Their interpretation has been defined already.

Complexity $n+1$: A proposition of complexity $n+1$ is a proposition of the form $\neg(\phi_1)$ or $\phi_1 \cdot \phi_2$, where $\cdot$ is a binary connector and ϕ_1 and ϕ_2 are propositions of complexity n. The interpretation of a proposition of complexity $n+1$ is computed by combining the interpretation of its components,

Example: For $\mathcal{I} = a\bar{b}c$, what is the interpretation of $a \to (b \wedge (c \vee a))$?

Truth tables

We often use **truth tables** to display the interpretations of a proposition. This is a simple visual way of checking whether a proposition is **universally true** (i.e. true for all interpretations).

a	b	$neg(a)$	$a \wedge b$	$a \vee b$	$a \rightarrow b$	$a \leftrightarrow b$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

Plan

1 Knowledge-based agents

2 What is logic

3 Propositional logic
- Syntax
- Semantics
- Logical equivalence
- Propositional inference: Enumeration method

Logical equivalence

Two propositions ϕ_1 and ϕ_2 are said to be **logically equivalent** if and only if they are true in the same models, i.e.:

$$\phi_1 \models \phi_2 \qquad \text{and} \qquad \phi_2 \models \phi_1$$

We write then $\phi_1 \equiv \phi_2$.

Substitution in formulas

We can easily check, for two propositional symbols α and β, that $(\alpha \vee \beta) \equiv (\beta \vee \alpha)$.

Substitution in formulas

We can easily check, for two propositional symbols α and β, that $(\alpha \vee \beta) \equiv (\beta \vee \alpha)$.

What if we substitute α with another proposition, say $\gamma \rightarrow \delta$?

This substitution maintains the logical equivalence: $((\gamma \rightarrow \delta) \vee \beta) \equiv (\beta \vee (\gamma \rightarrow \delta))$

Substitution in formulas

We can easily check, for two propositional symbols α and β, that $(\alpha \vee \beta) \equiv (\beta \vee \alpha)$.

What if we substitute α with another proposition, say $\gamma \rightarrow \delta$?

This substitution maintains the logical equivalence: $((\gamma \rightarrow \delta) \vee \beta) \equiv (\beta \vee (\gamma \rightarrow \delta))$

This result is more general: logical equivalence is conserved by substitution of subformulas.

Consequence: Many properties of $(\mathcal{PL}, \models)$ can be obtained by working on propositional symbols only *(i.e. working in Bool algebra)*.

Some important logical equivalences

Commutativity of $\wedge$:	$\alpha \wedge \beta \equiv \beta \wedge \alpha$
Commutativity of $\vee$:	$\alpha \vee \beta \equiv \beta \vee \alpha$
Associativity of $\wedge$:	$\alpha \wedge (\beta \wedge \gamma) \equiv (\alpha \wedge \beta) \wedge \gamma$
Associativity of $\vee$:	$\alpha \vee (\beta \vee \gamma) \equiv (\alpha \vee \beta) \vee \gamma$
Idempotence for $\wedge$:	$\alpha \wedge \alpha \equiv \alpha$
Idempotence for $\vee$:	$\alpha \vee \alpha \equiv \alpha$
Double negation elimination:	$\neg(\neg(\alpha)) \equiv \alpha$

Some important logical equivalences

Contraposition:	$\alpha \rightarrow \beta \wedge \alpha \equiv \neg\beta \rightarrow \neg\alpha$
Implication elimination:	$\alpha \rightarrow \beta \wedge \alpha \equiv \neg\alpha \vee \beta$
Equivalence elimination:	$\alpha \leftrightarrow \beta \wedge (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$
de Morgan's rules:	$\neg(\alpha \wedge \beta) \equiv (\neg\alpha) \vee (\neg\beta)$
de Morgan's rules:	$\neg(\alpha \vee \beta) \equiv (\neg\alpha) \wedge (\neg\beta)$
Distributivity of $\wedge$ over $\vee$:	$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$
Distributivity of $\vee$ over $\wedge$:	$\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$

Valid propositions (tautologies)

A proposition is called **valid** if it is true for all models. We also call such a proposition a **tautology**.

Example: The proposition $\alpha \vee \neg\alpha$ is a tautology.

Valid propositions (tautologies)

A proposition is called **valid** if it is true for all models. We also call such a proposition a **tautology**.

Example: The proposition $\alpha \vee \neg\alpha$ is a tautology.

Deduction theorem

For any propositions α and β, $\alpha \models \beta$ if and only if $\alpha \rightarrow \beta$ is a tautology.

Satisfiability

A proposition is called **satisfiable** if it is true in at least one model. We also call such a proposition a **tautology**.

Remarks:

- Proving the satisfiability of a proposition in $(\mathcal{PL}, \models)$ was the first problem proved to be NP-complete.
- α is valid if and only if $\neg\alpha$ is unsatisfiable.

Satisfiability

A proposition is called **satisfiable** if it is true in at least one model. We also call such a proposition a **tautology**.

Remarks:

- Proving the satisfiability of a proposition in $(\mathcal{PL}, \models)$ was the first problem proved to be NP-complete.
- α is valid if and only if $\neg\alpha$ is unsatisfiable.

Reductio ad absurdum (proof by refutation)

For any propositions α and β, $\alpha \models \beta$ if and only if the proposition $\alpha \wedge \neg\beta$ is unsatisfiable.

Plan

1 Knowledge-based agents

2 What is logic

3 Propositional logic
- Syntax
- Semantics
- Logical equivalence
- Propositional inference: Enumeration method

Principle

$KB \models \alpha$ means that all models of KB are models of α.

Consequently, to check whether $KB \models \alpha$, it is enough to compute the truth of KB and of α for all interpretations, and to check that the interpretations that make KB true also make α true.

Example: Given the knowledge base $KB = (A \vee C) \wedge (B \vee \neg C)$ and $\alpha = A \vee B$. Show that $KB \models \alpha$.